

3(a) — Command History

Question

To apply escape sequences, store command history, navigate previous/next commands, update input buffer, and test recall functionality.

```
GNU nano 8.7.1 skill3b.c *
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

char history[MAX_HISTORY][MAX_LENGTH];
int history_count = 0;
int current = -1;

void add_history(char *cmd)
{
    if (strlen(cmd) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], cmd);
        history_count++;
    }
    else
    {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }

    current = history_count;
}

void show_history()
{
    printf("\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
        printf("%d: %s\n", i + 1, history[i]);
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Ex

```

void previous_command()
{
    if (history_count == 0)
    {
        printf("No previous command.\n");
        return;
    }

    if (current > 0)
        current--;

    printf("Previous Command: %s\n", history[current]);
}

void next_command()
{
    if (history_count == 0)
    {
        printf("No next command.\n");
        return;
    }

    if (current < history_count - 1)
    {
        current++;
        printf("Next Command: %s\n", history[current]);
    }
    else
    {
        current = history_count;
        printf("Next Command: Empty\n");
    }
}

int main()
{

```

^G Help
 ^Y Exit

^O Write Out
 ^P Read File

^F Where Is
 ^\ Replace

^K Cut
 ^U Paste

```

printf("==== Command History System ====\\n");
printf("Type commands to store them.\\n");
printf("history - Show history\\n");
printf("prev    - Previous command\\n");
printf("next     - Next command\\n");
printf("exit     - Exit\\n");

while (1)
{
    printf("\\n$ ");

    if (fgets(command, MAX_LENGTH, stdin) == NULL)
        break;

    /* Remove newline */
    command[strcspn(command, "\\n")] = '\\0';

    if (strcmp(command, "exit") == 0)
        break;

    if (strcmp(command, "history") == 0)
    {
        show_history();
    }
    else if (strcmp(command, "prev") == 0)
    {
        previous_command();
    }
    else if (strcmp(command, "next") == 0)
    {
        next_command();
    }
    else
    {
        add_history(command);
    }
}

```

Output

```
compilation terminated.
sri_dhruthi@Dhruthi:~$ nano skill3a.c
sri_dhruthi@Dhruthi:~$ gcc skill3a.c -o skill3a
sri_dhruthi@Dhruthi:~$ ./skill3a
===== Command History System =====
Type commands to store them.
history - Show history
prev    - Previous command
next    - Next command
exit    - Exit

$ ls
Command stored: ls

$ pwd
Command stored: pwd

$ date
Command stored: date

$ exit
sri_dhruthi@Dhruthi:~$ |
```

3(b) — Dynamic Buffer and Linked List

Question

To allocate buffers dynamically, resize arrays, prevent buffer overflow, manage linked lists, release memory correctly, and verify with Valgrind.

```
GNU nano 2.7.1 SKILLSDDB.C *
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    int capacity = 2;
    int count = 0;
    int value;

    /* Dynamic buffer allocation */
    int *buffer = malloc(capacity * sizeof(int));

    if (buffer == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("==== Dynamic Buffer ==== \n");
    printf("Enter numbers (-1 to stop):\n");

    while (1)
    {
        printf("Enter value: ");
        scanf("%d", &value);

        if (value == -1)
            break;

        /* Resize buffer when full */
        if (count == capacity)
```

```
for (int i = 0; i < count; i++)
{
    struct Node *newNode =
        malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Node allocation failed.\n");
        break;
    }

    newNode->data = buffer[i];
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
        tail = newNode;
    }
    else
    {
        tail->next = newNode;
        tail = newNode;
    }
}

/* Display linked list */
struct Node *current = head;

printf("Linked List:\n");

while (current != NULL)
{
    printf("%d -> ", current->data);
    current = current->next;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^\ **Replace**

^K Cut
^U Paste

```

    }

    /* Display linked list */
    struct Node *current = head;

    printf("Linked List:\n");

    while (current != NULL)
    {
        printf("%d -> ", current->data);
        current = current->next;
    }

    printf("NULL\n");

    /* Free linked list memory */
    current = head;

    while (current != NULL)
    {
        struct Node *temp = current;
        current = current->next;
        free(temp);
    }

    /* Free dynamic buffer */
    free(buffer);

    printf("\nAll allocated memory released.\n");

    return 0;
}
|

```

Output

```
sri_dhruthi@Dhruthi:~$ nano skill3bb.c
sri_dhruthi@Dhruthi:~$ gcc -Wall -Wextra skill3bb.c -o skill3bb
sri_dhruthi@Dhruthi:~$ ./skill3bb
===== Dynamic Buffer =====
Enter numbers (-1 to stop):
Enter value: 10
Enter value: 20
Enter value: 30
Buffer resized to 4 elements.
Enter value: 40
Enter value: 50
Buffer resized to 8 elements.
Enter value: -1

Stored Values:
10 20 30 40 50

===== Linked List =====
Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> NULL

All allocated memory released.
sri_dhruthi@Dhruthi:~$ |
```